

DSA Algorithms Time Complexity Cheat Sheet

'*' Union-Find with path compression and union by rank has inverse Ackermann complexity $\alpha(n)$, effectively constant for practical inputs.

Algorithm / Operation	Best	Average	Worst
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$
DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$
BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$
Dijkstra (Heap)	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$
Bellman-Ford	$O(VE)$	$O(VE)$	$O(VE)$
Floyd-Warshall	$O(V^3)$	$O(V^3)$	$O(V^3)$
Kruskal	$O(E \log E)$	$O(E \log E)$	$O(E \log E)$
Prim (Heap)	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$
Topological Sort	$O(V+E)$	$O(V+E)$	$O(V+E)$
Union-Find (find/union)	$O(1)^*$	$O(\alpha(n))$	$O(\alpha(n))$
Trie Search	$O(L)$	$O(L)$	$O(L)$
Trie Insert	$O(L)$	$O(L)$	$O(L)$
KMP Search	$O(n+m)$	$O(n+m)$	$O(n+m)$
Rabin-Karp	$O(n+m)$	$O(n+m)$	$O(nm)$
Segment Tree Query	$O(\log n)$	$O(\log n)$	$O(\log n)$
Segment Tree Update	$O(\log n)$	$O(\log n)$	$O(\log n)$
Fenwick Tree Query	$O(\log n)$	$O(\log n)$	$O(\log n)$
Fenwick Tree Update	$O(\log n)$	$O(\log n)$	$O(\log n)$
Kadane's Algorithm	$O(n)$	$O(n)$	$O(n)$
Sliding Window	$O(n)$	$O(n)$	$O(n)$
Two Pointers	$O(n)$	$O(n)$	$O(n)$

Prefix Sum Query	$O(1)$	$O(1)$	$O(1)$
Prefix Sum Build	$O(n)$	$O(n)$	$O(n)$
Binary Exponentiation	$O(\log n)$	$O(\log n)$	$O(\log n)$
Euclidean GCD	$O(\log \min(a,b))$	$O(\log \min(a,b))$	$O(\log \min(a,b))$